VeriHealthi Embedded SDK 开发手册 (VESC2026)

REV: 1.0

VeriSilicon Confidential for VESC2026

芯原微电子（上海）股份有限公司

核对:

批准:

VERISILICON CONFIDENTIAL-NOT FOR REDISTRIBUTION

法律声明

COPYRIGHT INFORMATION

This document contains proprietary information of VeriSilicon Microelectronics (Shanghai) Co., Ltd. VeriSilicon reserves the right to make changes to any products herein at any time without notice. VeriSilicon does not assume any responsibility or liability arising out of the application or use of any product described herein, except as expressly agreed to in writing by VeriSilicon; nor does the purchase or use of a product from VeriSilicon convey a license under any patent rights, copyrights, trademark rights, or any other of the intellectual property rights of VeriSilicon or third parties.

DISCLOSURE/RE-DISTRIBUTION LIMITATIONS

The information contained herein is not to be used by or disclosed to third parties without the express written permission of an officer of VeriSilicon Microelectronics (Shanghai) Co., Ltd.

(VeriSilicon Distribution **Level D: CONFIDENTIAL – NOT FOR REDISTRIBUTION**).

TRADEMARK ACKNOWLEDGMENT

VeriSilicon® and the VeriSilicon logo design are the trademarks or the registered trademarks of VeriSilicon Microelectronics (Shanghai) Co., Ltd. All other brand and product names may be trademarks of their respective companies.

For our current distributors, sales offices, design resource centers, and product information, visit our web site located at https://www.verisilicon.com.

VeriSilicon Proprietary. Copyright © 2026 by VeriSilicon Microelectronics (Shanghai) Co., Ltd. All rights reserved.

VeriSilicon Confidential for VESC2026

文档修订历史

本节记录文档版本修订历史。

版本	日期	作者	说明
1.0	2026/5/7	Alex Lin	VESC2026 初稿

VeriSilicon Confidential for VESC2026

目录

法律声明 **2**

文档修订历史 **3**

目录 **4**

1. 概述 **5**

2. 术语 **5**

3. VERIHEALTHI 嵌入式 SDK 介绍 **6**

3.1 SDK 架构 6
3.2 SDK 功能模块 6

4. 驱动和 HAL **9**

4.1 HAL 设计目的 9
4.2 HAL 结构 9

5. VPI 功能介绍 **10**

5.1 EVENT 系统使用 10

6. OSAL 和 FREERTOS **12**

6.1 信号量使用 13
6.2 MUTEX 使用 13
6.3 消息队列使用 14
6.4 定时器使用 14

7. 统一的编程风格(CODING STYLE) **15**

8. 开发和调试 **16**

8.1 必备开发工具 16
8.2 导入工作空间配置 16
8.3 导入工程 16
8.4 编译工程 17
8.5 项目编译完成 18
8.6 调试工程 18

9. 注意事项 **20**

VeriSilicon Confidential for VESC2026

VERISILICON CONFIDENTIAL-NOT FOR REDISTRIBUTION

1. 概述

VeriHealthi 嵌入式 SDK 是芯原开发的 IoT 嵌入式软件基础。它不依赖具体的硬件平台，可以扩展支持越来越多的芯片和方案。SDK 使用者可以非常方便的调用提供的各种接口而无需关心底层的具体实现。同时 SDK 也提供了灵活简便的方式允许使用者基于 SDK 开发自有的驱动和接口。

SDK 提供了统一的 VPI (VeriSilicon Platform Interface)接口，里面包含了芯原开发的各类功能，大大简化了开发难度。客户基于 SDK 开发的上层软件可以适用于 Pegasus 芯片方案和将来其他新的芯片和方案而无需做大的改动。

2. 术语

术语	描述
IoT	Internet of Things，物联网
VS	VeriSilicon，芯原
VH	VeriHealthi，芯健康
SDK	Software Development Kit，软件开发包
OTA	Over The Air，空中升级技术
BLE	Bluetooth Low Energy，低功耗蓝牙
BSP	Board Support Package，板级支持包
VPI	VeriSilicon Platform Interface，芯原平台接口
OSAL	OS Abstraction Layer，操作系统抽象层
GPIO	General-purpose input/output，通用输入输出
UART	Universal Asynchronous Receiver/Transmitter，通用异步收发传输器
I2C	Inter-Integrated Circuit, I2C 接口
SPI	Serial Peripheral Interface，串行外设接口
I2S	Inter-IC Sound, I2S 接口
PDM	Pulse-Duration-Modulation, PDM 接口
DMA	Direct Memory Access，直接存储器访问
RAM	Random Access Memory，随机存储器
TCM	Tightly Coupled Memory，紧耦合内存
RTC	Real Time Clock，实时时钟
HAL	Hardware Abstraction Layer，硬件抽象层
SVT	System Vector Table，系统中断向量表
UVT	User Vector Table，用户中断向量表
ISR	Interrupt Service Routines，中断服务程序
LCF	Linker Control File，链接器控制文件
PPG	Photoplethysmography，光电传感器
ECG	Electrocardiogram，心电图
IMU	Inertial Measurement Unit，惯性测量装置
HR	Heart Rate，心率
RR	Respiratory Rate，呼吸率
SpO2	Oxygen Saturation，血氧饱和度
HRV	Heart Rate Variability，心率变异性
TMP	Temperature，温度
CRC	Cyclic Redundancy Check，循环冗余校验

VERISILICON CONFIDENTIAL-NOT FOR REDISTRIBUTION

3. VeriHealthi 嵌入式 SDK 介绍

3.1 SDK 架构

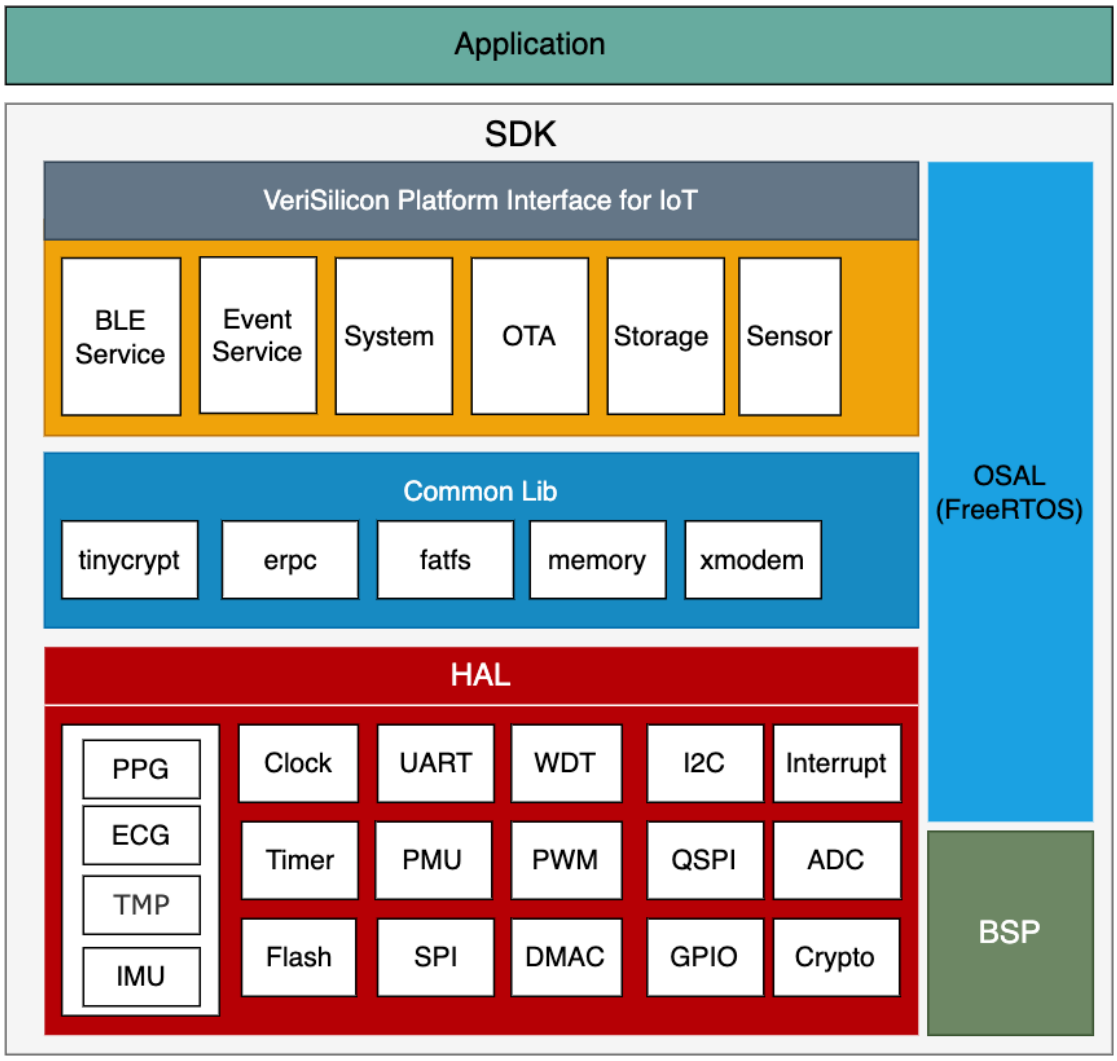


图 3-1 VeriHealthi 嵌入式 SDK 架构图

VeriHealthi 嵌入式 SDK 采用分层和模块化的架构设计思想。从宏观上看，整个应用可以划分为四大模块。

- OS & OSAL：操作系统和操作系统抽象层
- BSP：板级支持包
- 驱动和 HAL：硬件驱动和驱动抽象层
- 模块和 VPI：功能模块和芯原平台接口

3.2 SDK 功能模块

3.2.1 OS & OSAL

该软件平台提供了实时操作系统的支持，默认使用 FreeRTOS。

实时操作系统提供了强大，高性能，高实时性的接口，使得嵌入式的开发变的非常的高效，容易和可靠。

OSAL 是操作系统抽象层。该层的目的是解耦系统软件和操作系统，使得整个软件平台不依赖特定的操作系统。软件平台因此可以非常方便的扩展支持其他优秀的实时操作系统而无需修改驱动，功能模块和上层应用。

OSAL 的功能列表：

VERISILICON CONFIDENTIAL-NOT FOR REDISTRIBUTION

功能名称	描述	典型接口
osal_event	事件和消息	osal_create_event_queue, osal_send_event, osal_wait_event
osal_task	任务管理	osal_create_task, osal_sleep, osal_start_schedule, osal_end_schedule
osal_heap	内存管理	osal_malloc, osal_free osal_malloc_noncache osal_free_noncache
osal_notify	通知	osal_task_notify, osal_task_notify_from_isr, osal_task_notify_wait
osal_lock	同步锁	osal_create_mutex, osal_unlock_mutex, osal_lock_mutex, osal_delete_mutex
osal_semaphore	信号量	osal_create_sem, osal_create_counting_sem, osal_sem_wait, osal_sem_post, osal_sem_post_isr
osal_time	时间，获得系统启动后的运行时间	osal_get_uptime
osal_sys_state	状态	osal_dump_heap_size osal_get_task_name
osal_queue	消息队列	osal_create_queue, osal_send_queue, osal_send_queue_isr, osal_receive_queue osal_delete_queue
osal_timer	定时器	osal_timer_create, osal_timer_start, osal_timer_start_isr, osal_timer_reset, osal_timer_reset_isr, osal_timer_stop, osal_timer_stop_isr, osal_timer_delete
osal_spinlock	自旋锁	osal_create_spinlock, osal_lock_spinlock, osal_unlock_spinlock, osal_delete_spinlock

3.2.2 BSP

BSP 包含三大部分：

- SoC 芯片相关功能，芯片初始化各个组件的入口，管理对应芯片的 pinmux，独有功能和配置，包括各个 IP 的寄存器地址分配。
  延时函数： void delay_us(uint32_t us)
- ARCH CPU 架构相关功能，包括 RTOS 移植代码(port.c)和配置，以及 CPU/ZSP 提供的私有接口，包括 sleep，halt 和 idle 等低功耗接口。
- Board 板子相关功能，板子初始化各个外设的入口，管理所有板载设备。 提供 board_register 和 board_init 两大主要功能。

3.2.3 驱动和 HAL

驱动： 包含芯片各个 IP 的驱动以及外设驱动。

VERISILICON CONFIDENTIAL-NOT FOR REDISTRIBUTION

HAL：提供统一的抽象层，上层使用 HAL 接口来间接调用驱动接口。

3.2.4 模块和 VPI

模块功能：提供各种通用型模块的实现，包括 BLE 服务，Event 系统以及内存管理等模块。
VPI：各个模块功能向上层提供统一的 VPI 接口用来提供服务。

VeriSilicon Confidential for VESC2026

VERISILICON CONFIDENTIAL-NOT FOR REDISTRIBUTION

4. 驱动和 HAL

4.1 HAL 设计目的

目标

- 硬件驱动和上层应用解耦
- 芯片 IP 驱动可复用
- 外部设备驱动可复用

核心设计

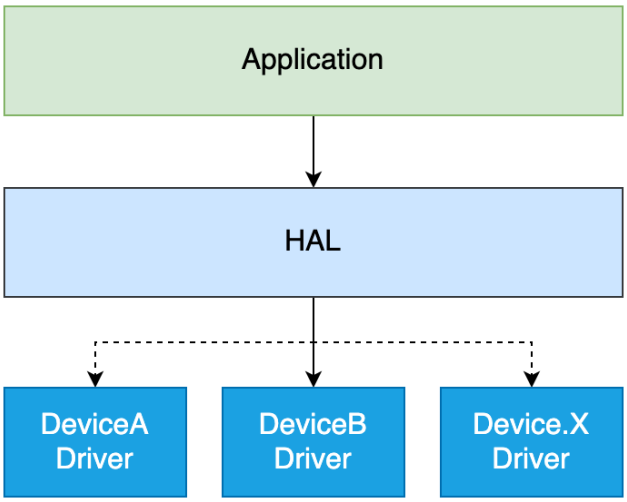
- 硬件驱动抽象化
- 上层应用和驱动无耦合
- 驱动通过设备注册方式和 HAL 建立关联

所有芯片 IP 模块驱动和外设驱动都遵循 HAL 设计。

4.2 HAL 结构

HAL 层非常的薄，简单理解就是函数指针的调用。

HAL 接口可以跟任何一个匹配的驱动链接上，甚至可以链接多个不同的设备驱动。



VeriSilicon Confidential for VESC2026

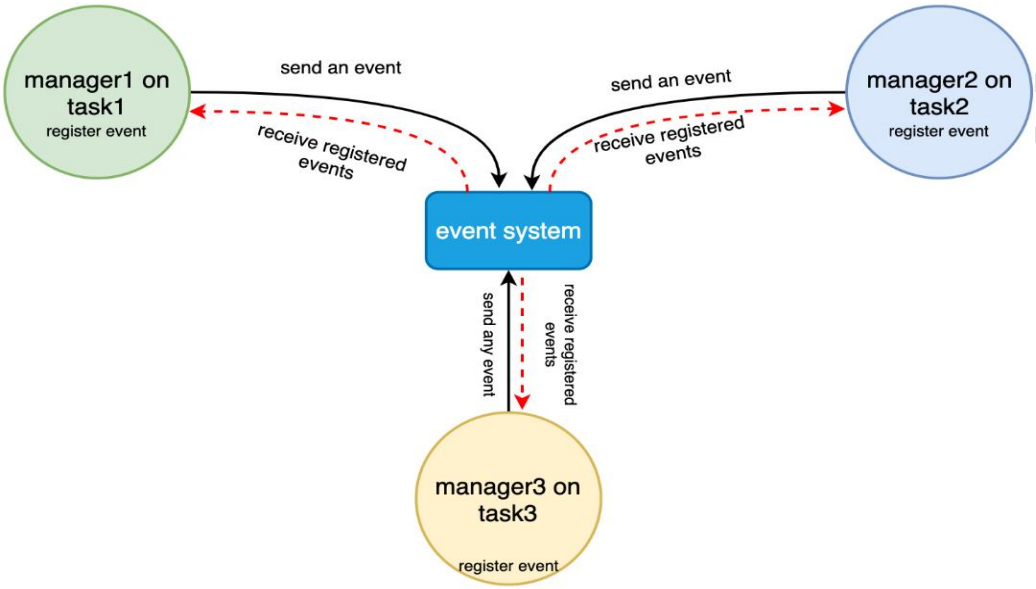
VERISILICON CONFIDENTIAL-NOT FOR REDISTRIBUTION

5. VPI 功能介绍

5.1 Event 系统使用

5.1.1 基础框架设计

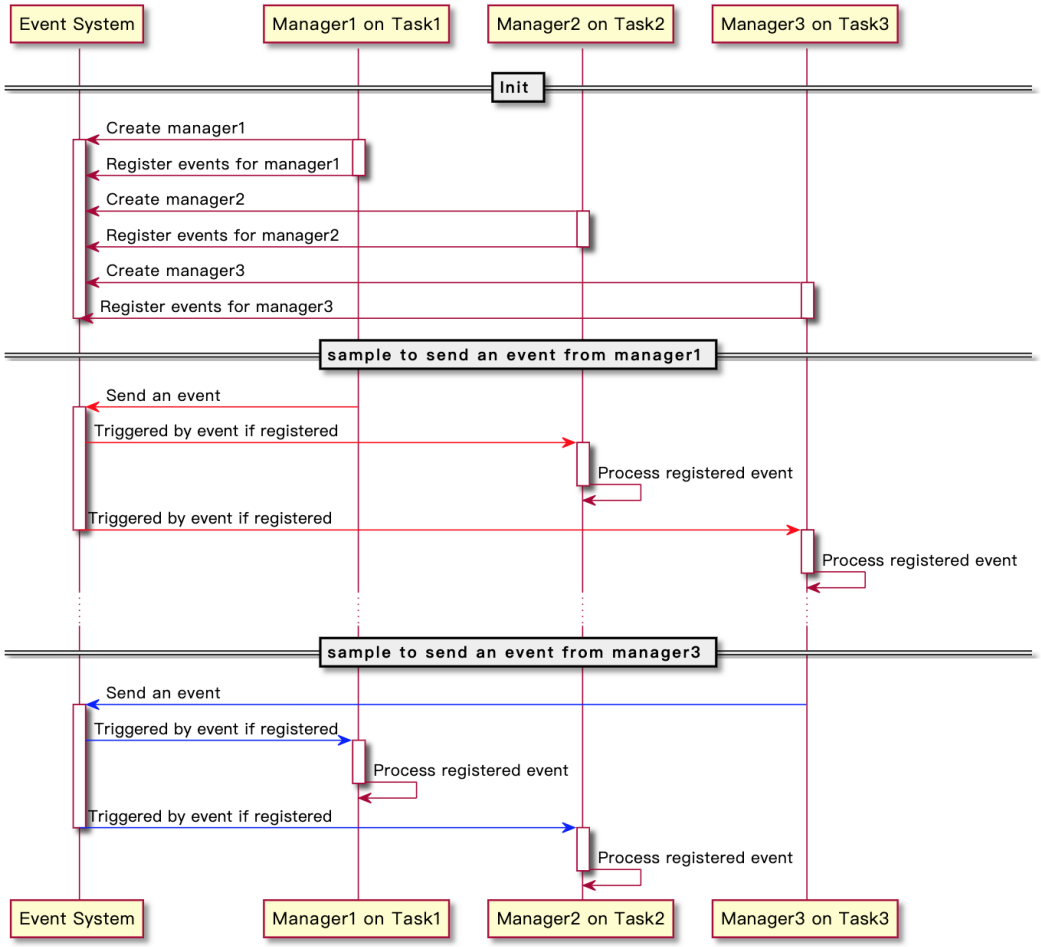
- 一个 task 对应一个 manager
- Manager 可以注册感兴趣的 event，一个 manager 可以注册多个 event
- Manager 发一个 event，所有注册过该 event 的 manager 都会收到
- 需要在 task 的 while(1) loop 里调用 vpi_event_listen 来监听其他 manager 发来的 event
- 使用 vpi_event_notify 接口给其他 manager 发送 event



VeriSilicon Confidential
for VESC2026

VERISILICON CONFIDENTIAL-NOT FOR REDISTRIBUTION

5.1.2 Event 系统使用例子



阶段	步骤	动作
初始化	1	创建多任务
	2	每个任务创建各自的 manager
	3	新建的 manager 调用 Event System 来注册关心的 Events
工作	1	Manager1 广播一个 event 信息
	2.1	Manager2 收到该 event 广播，对应处理
	2.2	Manager3 收到该 event 广播，对应处理

5.1.3 Event 接口

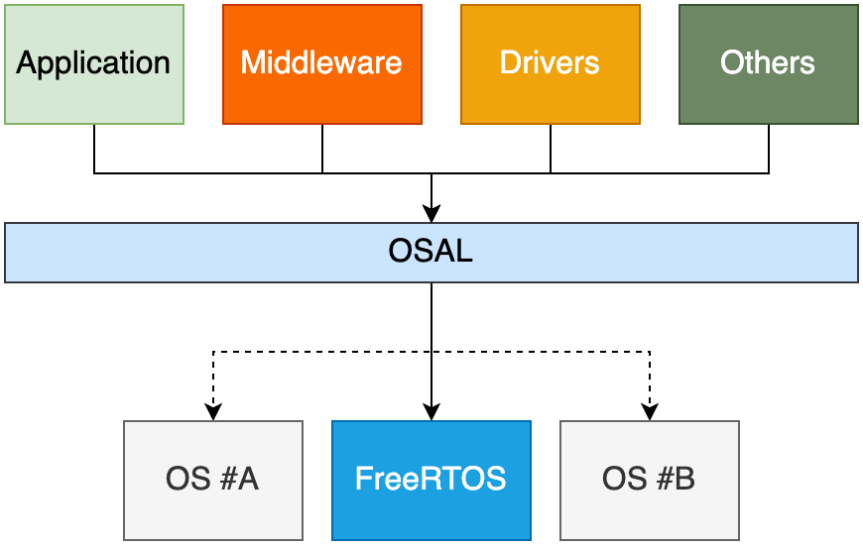
Event 函数	功能描述
vpi_event_set_queue_size	初始化事件系统的事件队列大小（必须在调用 vpi_event_register 之前调用，事件队列大小默认为 10）
vpi_event_new_manager	新建 manager，manager_id 为枚举值；handler 类型为函数指针；每次调用内部都会分配内存用于保存新 manager，禁止重复相同调用，以免内存造成泄漏
vpi_event_new_group	新建 group，用于在一个 task 里容纳多个独立功能的 manager
vpi_event_add_to_group	将 manager 添加到 group 里
vpi_event_register	Manager 注册 event，event 类型为枚举值，可自定义扩展
vpi_event_unregister	Manager 注销 event
vpi_event_notify	Manager 广播 event，用于发送一个消息给其它 manager
vpi_event_notify_from_isr	Manager 广播 event，用于从中断中发送一个消息给其它 manager
vpi_event_listen	监听其它 manager 发送的消息；需要在 while(1) loop 里执行

VERISILICON CONFIDENTIAL-NOT FOR REDISTRIBUTION

6. OSAL 和 FreeRTOS

OSAL (OS Abstraction Layer)

- 抽象化 OS，使得软件实现和 OS 无关
- 可以非常方便的扩展支持更多 OS
- 当前默认支持 FreeRTOS



相关接口定义

OSAL 接口	功能描述
osal_task	任务管理，包括任务创建，销毁等功能。任务优先级范围从 0 到 7；7 为最高优先级。
osal_event, osal_notify	事件和通知功能
osal_malloc, osal_free	堆动态分配和释放，该接口分配的内存支持 Cache 操作，性能更高，但需要考虑内存一致性问题。如果只是单个 CPU 访问内存，推荐使用该接口
osal_malloc_noncache, osal_free_noncache	堆动态分配和释放，该接口分配的内存不支持 Cache 操作；DMA 或者其它 CPU 可以直接共享内存而无需要考虑内存一致性问题
osal_create_mutex, osal_lock_mutex, osal_unlock_mutex, osal_delete_mutex	安全锁。锁的操作要求先调用 create 接口做创建和初始化，后续再调用 lock/unlock 接口。 禁止在中断处理函数里使用 mutex 接口。
osal_create_spinlock, osal_delete_spinlock, osal_unlock_spinlock, osal_lock_spinlock	自旋锁。锁的操作要求先调用 create 接口做创建和初始化，后续再调用 lock/unlock 接口。与 mutex 不同，自旋锁在获取锁失败时不会使任务进入阻塞或休眠状态，而是让 CPU 处于忙等（Spinning）状态直至获取锁。
osal_create_sem, osal_create_counting_sem, osal_sem_post, osal_sem_post_isr, osal_sem_wait	信号量。信号量的操作要求先调用 create 接口做创建和初始化，使用 osal_create_sem 创建二值信号量，使用 osal_create_counting_sem 创建计数信号量，后续再调用 post 和 wait 接口。 wait 会使 task 处于阻塞状态一段时间（根据设置的 timeout）； 在中断里需要使用 osal_sem_post_isr 接口进行发送。
osal_create_queue, osal_send_queue, osal_send_queue_isr, osal_receive_queue, osal_delete_queue	消息队列。消息队列的操作要求先调用 create 接口做创建和初始化，后续再调用 send 和 receive 接口。 receive 会使 task 处于阻塞状态一段时间（根据设置的 timeout）； 在中断里需要使用 osal_send_queue_isr 接口进行发送

VERISILICON CONFIDENTIAL-NOT FOR REDISTRIBUTION

osal_timer_create, **osal_timer_start,** **osal_timer_start_isr,** **osal_timer_reset,** **osal_timer_reset_isr,** **osal_timer_stop,** **osal_timer_stop_isr,** **osal_timer_delete**	定时器。定时器的操作要求先调用 create 接口做创建和初始化，后续再调用 start, stop 和 reset 接口。 start, stop 和 reset 会使 task 处于阻塞状态一段时间（根据设置的 timeout）； 在中断里需要使用 osal_timer_xxxx_isr 接口进行操作

- 注意事项
  - osal_malloc 和 osal_malloc_noncache 会从 OS 管理的堆空间里分配内存，而 malloc 的内存分配由 libc 实现，从系统另外的空间分配。因此 osal_malloc 和 osal_free，osal_malloc_noncache 和 osal_free_noncache，以及 malloc 和 free 需要配对使用。嵌入式开发默认需要使用 osal 接口来进行内存分配；OS 起来之前分配使用 malloc/free；
  - FreeRTOS 其它接口可以直接使用；

6.1 信号量使用

信号量一般可用于阻塞等待一个事件（即信号量），支持从中断处理函数或者非中断代码发送信号量。当需要任务同步和互斥访问共享资源时，使用 osal_create_sem 创建二值信号量，当允许多个任务同时访问有限数量的资源时，使用 osal_create_counting_sem 创建计数信号量。

典型的使用流程：

```
OsalSemaphore *sem;

void demo_isr(void)
{
    osal_sem_post_isr(sem);
}

void demo_task1(void)
{
    osal_sem_post(sem);
}

void demo_task2(void *param)
{
    osal_create_sem(&sem);
    while (1) {
        osal_sem_wait(sem, OSAL_WAIT_FOREVER);
        /* do some thing */
    }
}
```

6.2 Mutex 使用

Mutex 同步锁可用于保护多 task 同时访问时对于数据或者功能的保护。
注意：该接口不适用于中断处理函数内调用。
典型使用流程：

- 创建和初始化 mutex 实例：osal_create_mutex(OsalMutex **mu), 其中 mu 的内存由调用该接口的函数负责分配，因此重复调用该接口需要注意内存泄漏问题，在合适的地方调用 osal_delete_mutex 进行内存释放；
- 在需要保护的数据或者功能保的开始处调用 osal_lock_mutex 实现获得同步锁操作，如果有其它 task 调用过该接口并使用了相同的 mutex 作为参数，且已经获得该同步锁，那么当前 task 会进入等待锁状态，一直到对方释放了该锁才会继续；
- 在需保护的数据或者功能结束处调用 osal_unlock_mutex 用于释放锁；
- 若该锁不再需要，调用 osal_delete_mutex 进行销毁操作；为了效率和稳定，请勿重复创建/销毁相同功能的 mutex，一直复用该 mutex 即可。

VERISILICON CONFIDENTIAL-NOT FOR REDISTRIBUTION

6.3 消息队列使用

消息队列一般可用于阻塞等待一个消息，支持从中断处理函数或者非中断代码发送队列。

典型的使用流程：

```
OsalQueue *queue;
uint8_t send_data[10];

void demo_isr(void)
{
    osal_send_queue_isr(queue, send_data);
}

void demo_task1(void)
{
    osal_send_queue(queue, send_data, 10);
}

void demo_task2(void *param)
{
    uint8_t receive_data[10];
    osal_create_queue(&queue, 2, 10);
    while (1) {
        osal_receive_queue(queue, receive_data, OSAL_WAIT_FOREVER);
        /* do some thing */
    }
}
```

6.4 定时器使用

定时器可以用于一些定时任务，支持从中断处理函数或者非中断代码操作定时器。

典型的使用流程：

```
OsalTimer *timer;

void demo_isr(void)
{
    osal_timer_stop_isr(timer);
}

void demo_timer_cb(OsalTimer *timer, void *param)
{
    /* do some thing */
}

void demo_task2(void *param)
{
    uint8_t param;
    osal_timer_create(&timer, "demo_timer", 1000, true, demo_timer_cb,          &param);
    osal_timer_start(timer, OSAL_WAIT_FOREVER);
}
```


VERISILICON CONFIDENTIAL-NOT FOR REDISTRIBUTION

7. 统一的编程风格(Coding Style)

基本要求

要求	风格说明	例子
文件格式	Unix 格式文件	无
缩进	4 个空格缩进，禁止使用 tab	int abc;
行长度	每一行代码不超过 80 个字符	无
结构体/枚举命名	遵循大驼峰式命名法	VpiHwCtx
函数命名	采用 sb_do_sth 的规则	sensor_read_samples
变量名称	使用小写字符，字符中间用'_'分割	reg_value
枚举元素	定义使用全大写的命名规则	VPE_CMD_BASE

相关工具

- clang-format
- SDK 提供的.clang-format 规则文件(位于项目根目录下)

数据结构体示例

```
typedef struct VpiHwCtx {
    char device_name[32];
    int hw_context;
    int task_id;
    int priority;
} VpiHwCtx;
```

枚举定义示例

```
typedef enum VpeCmd {
    VPE_CMD_BASE,
    VPE_CMD_DEC_PIC_CONSUME,
    VPE_CMD_DEC_STRM_BUF_COUNT,
    /*hw downloader command*/
} VpeCmd;
```

```
static int init_input_threads(void)
{
    int i, ret;

    for (i = 0; i < nb_input_files; i++) {
        ret = init_input_thread(i);
        if (ret < 0)
            return ret;
    }
    return 0;
}
```


VERISILICON CONFIDENTIAL-NOT FOR REDISTRIBUTION

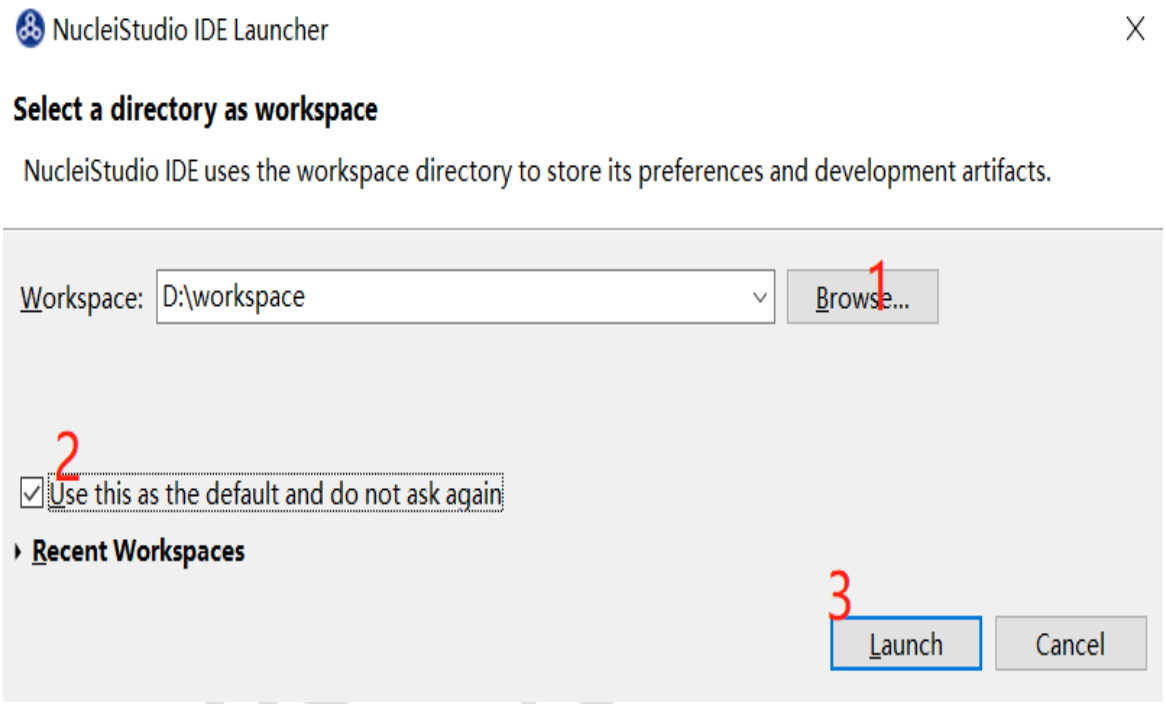
8. 开发和调试

8.1 必备开发工具

工具	详细描述
Windows PC	Windows PC，推荐 Windows 11 系统
NucleiStudio IDE	Nuclei Studio IDE 2024.02 版本

8.2 导入工作空间配置

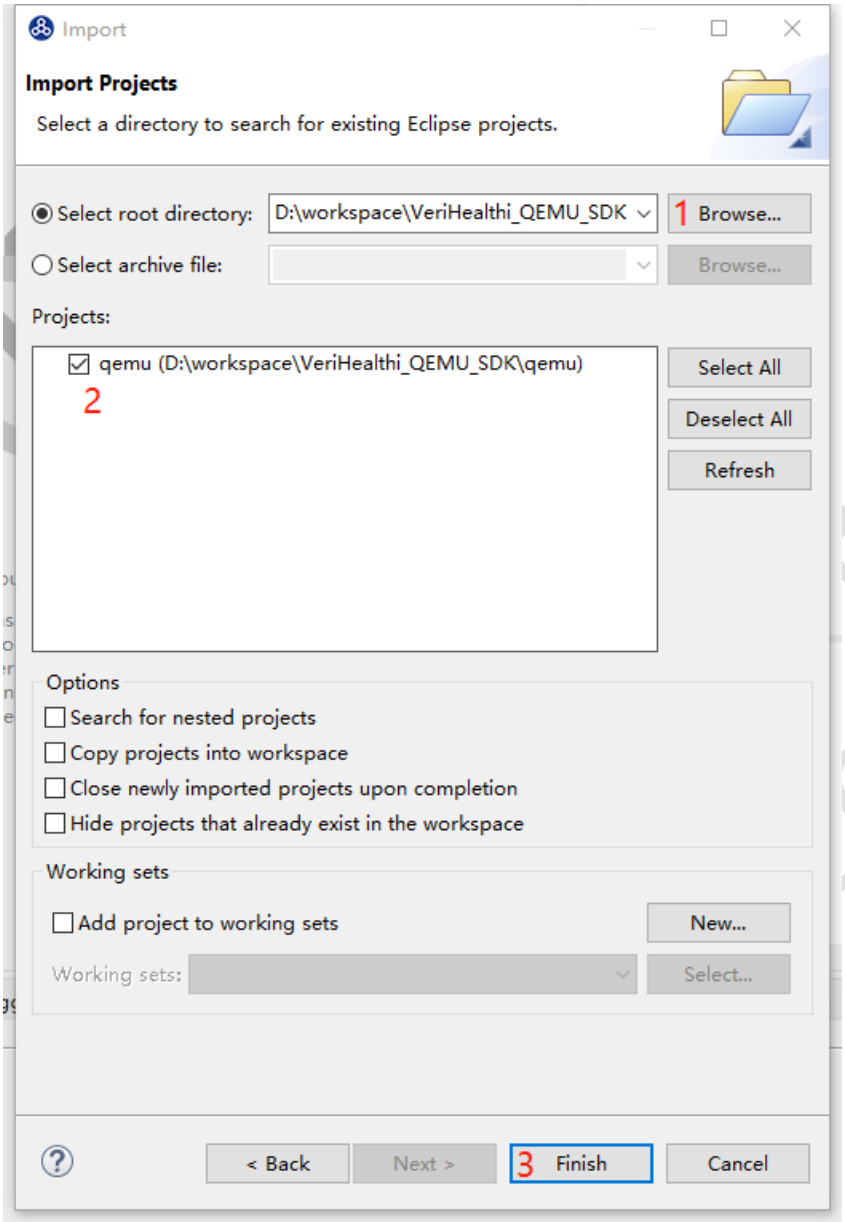
- 双击 NucleiStudio.exe
- 在弹出的 NucleiStudio IDE Launcher 界面
  - 点击 Browse，选择工作空间目录来存储工程文件
  - 选择 Use this as the default and do not ask again
  - 点击 Launch，进入工作空间导入工程



8.3 导入工程

- 在 NucleiStudioIDE 软件的左上角，依次选择：File->Import->General->Existing Projects into Workspace->Next
- 在弹出的 Import 界面
  - 点击 Browse，选择 qemu 工程所在的文件夹
  - 在 Projects，选择 qemu
  - 点击 Finsh，导入工程

VERISILICON CONFIDENTIAL-NOT FOR REDISTRIBUTION



8.4 编译工程

- 点击 Restore，进入工程窗口

VERISILICON CONFIDENTIAL-NOT FOR REDISTRIBUTION

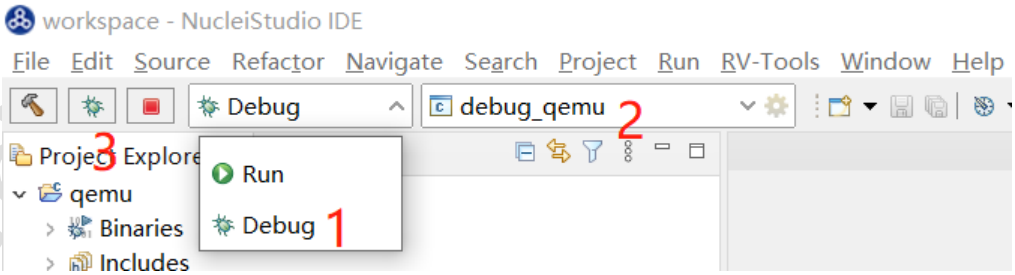
- 首次导入工程需右键 qemu，选择 Clean Project
- 展开目录 qemu->galaxy_sdk，找到 main.c
- 右键 main.c，选择 Build Project，编译开始

8.5 项目编译完成

```
Problems  Tasks  Console  Properties
CDT Build Console [qemu]

Building target: qemu.out
Invoking: GNU RISC-V Cross C++ Linker
riscv-nuclei-elf-g++ -march=rv32imafc -mabi=ilp32f -mtune=nuclei-300-series -mcmodel=medlow -mno-save-restore -O2 -ffunction-sections -fdata-sections -fno-
common -Werror -Wall  -g -T "D:\workspace\CocopalmRLibQemuSDK2.0\CocopalmRLibQemuSDK\galaxy_sdk\n309_iot_qemu.ld" -nostartfiles -Xlinker --gc-sections -L"D:
\workspace\CocopalmRLibQemuSDK2.0\CocopalmRLibQemuSDK\galaxy_sdk\bsp\lib" -L"D:\workspace\CocopalmRLibQemuSDK2.0\CocopalmRLibQemuSDK\galaxy_sdk\drivers\lib"
-L"D:\workspace\CocopalmRLibQemuSDK2.0\CocopalmRLibQemuSDK\galaxy_sdk\modules\lib" -L"D:\workspace\CocopalmRLibQemuSDK2.0\CocopalmRLibQemuSDK\galaxy_sdk\os
\lib" -L"D:\workspace\CocopalmRLibQemuSDK2.0\CocopalmRLibQemuSDK\galaxy_sdk\osal\lib" -Wl,-Map,"qemu.map" --specs=nano.specs --specs=nosys.specs -o
"qemu.out" -Wl,--start-group  ./galaxy_sdk/drivers/src/hal_adc.o ./galaxy_sdk/drivers/src/hal_clock.o ./galaxy_sdk/drivers/src/hal_codec.o
./galaxy_sdk/drivers/src/hal_crc.o ./galaxy_sdk/drivers/src/hal_crypto.o ./galaxy_sdk/drivers/src/hal_dac.o ./galaxy_sdk/drivers/src/hal_dmac.o
./galaxy_sdk/drivers/src/hal_ecg.o ./galaxy_sdk/drivers/src/hal_eeprom.o ./galaxy_sdk/drivers/src/hal_flash.o ./galaxy_sdk/drivers/src/hal_gpd.o
./galaxy_sdk/drivers/src/hal_gpio.o ./galaxy_sdk/drivers/src/hal_i2c.o ./galaxy_sdk/drivers/src/hal_i2s.o ./galaxy_sdk/drivers/src/hal_i3c.o
./galaxy_sdk/drivers/src/hal_imu.o ./galaxy_sdk/drivers/src/hal_mbox.o ./galaxy_sdk/drivers/src/hal_otp.o ./galaxy_sdk/drivers/src/hal_pdm.o
./galaxy_sdk/drivers/src/hal_pmu.o ./galaxy_sdk/drivers/src/hal_ppg.o ./galaxy_sdk/drivers/src/hal_pwm.o ./galaxy_sdk/drivers/src/hal_rtc.o
./galaxy_sdk/drivers/src/hal_spi.o ./galaxy_sdk/drivers/src/hal_timer.o ./galaxy_sdk/drivers/src/hal_tmp.o ./galaxy_sdk/drivers/src/hal_uart.o
./galaxy_sdk/drivers/src/hal_usb.o ./galaxy_sdk/drivers/src/hal_vdt.o ./galaxy_sdk/drivers/src/hal_wdt.o  ./galaxy_sdk/bsp/src/intexc_riscv.o
./galaxy_sdk/bsp/src/newlib.o ./galaxy_sdk/bsp/src/portasm.o ./galaxy_sdk/bsp/src/qemu_board.o ./galaxy_sdk/bsp/src/startup_riscv.o
./galaxy_sdk/event_sample.o ./galaxy_sdk/main.o   -lstdc++ -lbsp_riscv -ldriver_riscv -lcommon_riscv -losal_riscv -los_riscv -Wl,--end-group
Finished building target: qemu.out

Invoking: GNU RISC-V Cross Create Flash Image
riscv-nuclei-elf-objcopy -O ihex "qemu.out"  "qemu.hex"
Invoking: GNU RISC-V Cross Create Listing
riscv-nuclei-elf-objdump --source --all-headers --demangle --line-numbers --wide "qemu.out" > "qemu.lst"
Invoking: GNU RISC-V Cross Print Size
riscv-nuclei-elf-size --format=berkeley "qemu.out"
Finished building: qemu.hex

   text    data     bss     dec     hex filename
  21310   33120   23544   77974   13096 qemu.out
Finished building: qemu.siz
Finished building: qemu.lst


18:53:11 Build Finished. 0 errors, 0 warnings. (took 10s.731ms)
```

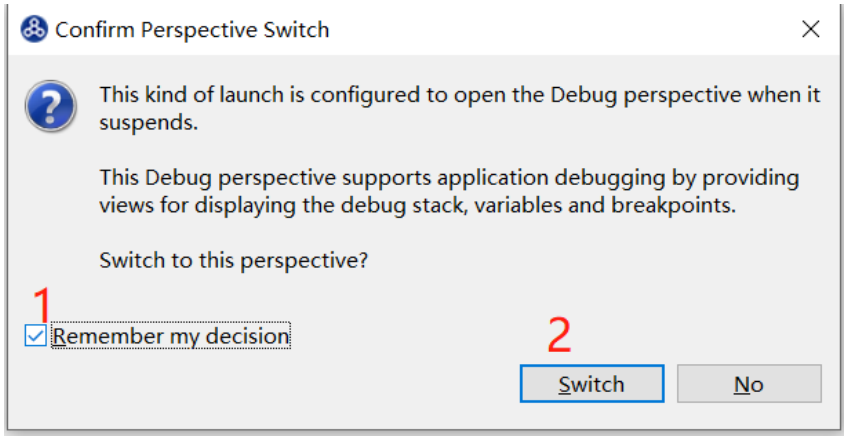
8.6 调试工程

- 按下开发板 reset(SW2)按键（QEMU 平台无需这一步操作）
- 选择 Launch Mode 为“Debug”
- 选择 Launch Configuration 为“cocopalmR_debug_openocd”（QEMU SDK 请选择“debug_qemu”）
- 点击 Launch in ‘Debug’ mode 启动调试

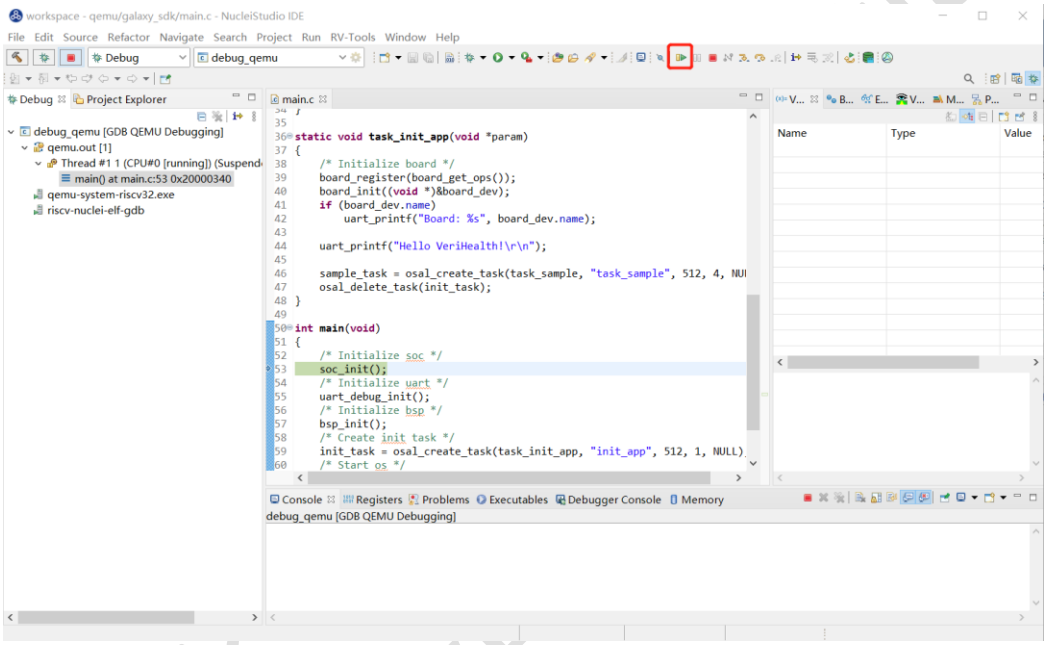


- 初次启动调试后会弹出 Confirm Perspective Switch 弹窗
  - 勾选 Remember my decision
  - 点击 Switch

VERISILICON CONFIDENTIAL-NOT FOR REDISTRIBUTION

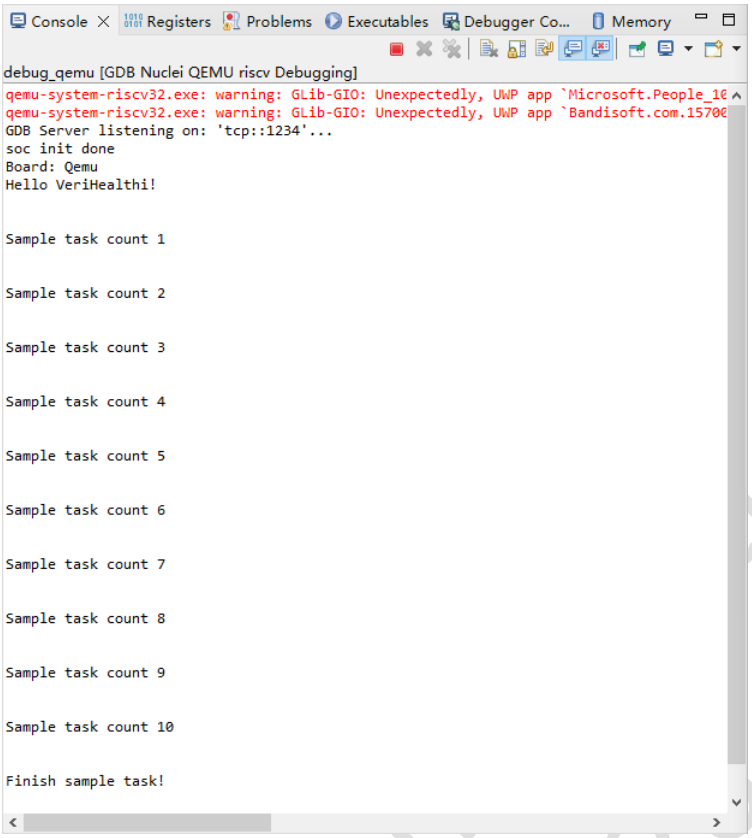


- 调试成功后，跳到如下界面，点击 Resume，运行程序



- 运行成功会在 Console 界面打印如下内容

VERISILICON CONFIDENTIAL-NOT FOR REDISTRIBUTION



9. 注意事项

注意事项
SDK 项目编译默认会将 warning 判定为 error。遇到该问题需要解决代码 warning 问题，或者根据官网 FAQ 页面来修改项目设置
SDK 目前没有包含文件系统，所以无法调用 fopen。嵌入式系统一般不需要 fopen，因此 SDK 里没有实现 fopen，可使用静态数组的方式导入数据
编译若遇到代码 overlap 的错误提示，需要合理调整 RAM 分配；对于只读的数组需要加上 const，这样编译链接的时候会将该数组分配到 flash 空间，避免过多占用 RAM 空间

VeriSilicon Confidential for VESC2026

VERISILICON CONFIDENTIAL-NOT FOR REDISTRIBUTION